

Model constants (fixed packet input)

Constant	Value	Meaning
CAPACITY_A	5700	cell A charge units
CAPACITY_B	6300	cell B charge units (manufacturing variance)
CC_CURRENT	50	constant-current phase current
CV_VOLTAGE_THRESHOLD	4150	mV; CC->CV transition point
CV_DECAY_STEP	1.2	per tick, CV regulation current
TAPER_CURRENT_THRESHOLD	8	CV->TAPER transition point
TAPER_CURRENT	3	fixed TAPER-mode current
IR_DROP_PER_UNIT	2.0	mV per unit delivered current
BASE_VOLTAGE	3700	mV, at soc=0
VOLT_PER_SOC_PCT	6.0	mV per 1% state of charge
AMBIENT	250	0.1 degC units, starting/ambient temp
HEAT_FULL	6	0.1 degC/tick, full-rate charging
HEAT_DERATED	3	0.1 degC/tick, derated charging
HEAT_TAPER	1	0.1 degC/tick, TAPER mode (always)
COOL_PER_TICK	2	0.1 degC/tick, passive cooling
DERATE_TEMP	450	0.1 degC; derating engages at/above
DERATE_FACTOR	0.5	multiplier on mode current
FAULT_TEMP_HIGH	575	0.1 degC; fault latches at/above
FAULT_TEMP_LOW	520	0.1 degC; release eligible at/below
FAULT_RELEASE_TICKS	4	consecutive cool ticks to release
OVERCURRENT_MAX	45	hard clamp, applied after derating
BALANCE_THRESHOLD	1.5	%; SoC gap that triggers balancing
BALANCE_BLEED	8	units diverted from the leading cell

Figure 1 -- Model constants.

Per-tick update rules, in order (applied every tick, $t=1..320$)

1. Mode transition (shared, pack-level): CC->CV if $\max(\text{PREVIOUS tick's voltage_a, voltage_b}) \geq \text{CV_VOLTAGE_THRESHOLD}$. CV->TAPER if CV regulation current $\leq \text{TAPER_CURRENT_THRESHOLD}$. One-way only.
2. Mode-commanded current: $\text{CC_CURRENT} / \text{CV regulation current} / \text{TAPER_CURRENT}$.
3. CV taper-down: in CV mode only, after step 2, reduce CV regulation current by CV_DECAY_STEP (floor 0). Unconditional -- not gated by fault status (see section 2).
4. Thermal derating and fault gating -> BASE current (shared): if fault latched, $\text{base}=0$. Else if $\text{INCOMING temp} \geq \text{DERATE_TEMP}$, $\text{base} = \text{mode-current} * \text{DERATE_FACTOR}$ (every mode, TAPER included). Else $\text{base} = \text{mode-current}$ unmodified.
5. Overcurrent clamp: cap BASE current at OVERCURRENT_MAX . Applied AFTER derating, before balancing.
6. CELL BALANCING (per cell): $\text{diff} = \text{soc_a} - \text{soc_b}$, using EACH cell's own PREVIOUS-tick SoC. If $\text{base}>0$ and $|\text{diff}| \geq \text{BALANCE_THRESHOLD}$: the HIGHER-SoC cell gets $\max(0, \text{base} - \text{BALANCE_BLEED})$; the other cell gets the full base current, unchanged. Otherwise both cells get the same base current. Re-evaluated every tick -- compute the comparison fresh, do not hardcode a cell (for this packet's fixed capacities the same cell ends up ahead for the whole run).
7. Heat generation (shared, driven by BASE current, not either cell's post-balancing current): 0 if $\text{base}=0$. Else, in TAPER mode: ALWAYS HEAT_TAPER , regardless of temperature. Else if $\text{incoming temp} \geq \text{DERATE_TEMP}$: HEAT_DERATED . Else: HEAT_FULL .
8. Temperature update: $\text{temp} = \text{incoming temp} + \text{heat} - \text{COOL_PER_TICK}$.
9. Fault-latch hysteresis: latch if not already latched and $\text{NEW temp} \geq \text{FAULT_TEMP_HIGH}$. Release only after $\text{FAULT_RELEASE_TICKS}$ CONSECUTIVE ticks at/below FAULT_TEMP_LOW -- one tick back above resets the counter to 0.
10. State of charge (per cell): $\text{soc_x} = \min(100, \text{soc_x_prev} + \text{current_x} / \text{CAPACITY_X} * 100)$ -- each cell uses its OWN capacity.
11. Terminal voltage (per cell): $\text{voltage_x} = \text{BASE_VOLTAGE} + \text{soc_x} * \text{VOLT_PER_SOC_PCT} - \text{current_x} * \text{IR_DROP_PER_UNIT}$. Each cell's own voltage feeds step 1's NEXT-tick $\max()$ check.

Figure 2 -- Per-tick update rules, in order.

Figure 3 worked example (fake data)

Certificate format (S03), worked example with FAKE numbers. These numbers do not correspond to any real tick of this model, and deliberately do NOT satisfy this packet's own formulas (clamp, voltage, etc) -- they illustrate the TEXT FORMAT only. Never use them to sanity-check or validate your own implementation's logic:

```
CELLGUARD10-CERT-V2
t=1;mode=CC;soc_a=12.34;soc_b=11.90;temp=317;current_a=50.00;
  current_b=50.00;voltage_a=3850.50;voltage_b=3846.10;fault=False
t=2;mode=CC;soc_a=12.67;soc_b=12.05;temp=321;current_a=42.00;
  current_b=50.00;voltage_a=3852.50;voltage_b=3847.20;fault=False
...
t=320;mode=TAPER;soc_a=77.10;soc_b=76.55;temp=288;current_a=3.00;
  current_b=3.00;voltage_a=4150.90;voltage_b=4145.10;fault=False
```

(The two-space-indented continuation above is only this figure wrapping for page width -- the actual certificate has NO line break there; see the one-line rule below.)

Rules: one line per tick, t=1 through t=320, in order, with no internal line break regardless of length. soc_a/soc_b/current_a/current_b/voltage_a/voltage_b are ALWAYS shown with exactly two digits after the decimal point -- 50.00, never 50 or 50.0. temp is an integer (0.1 degC units). fault is True or False. Certificate hash: SHA-256 of the full text (header through the t=320 line, newline-joined, trailing newline included), first 16 hex characters. (Tick 2's example shows current_a != current_b -- balancing is active that tick, bleeding cell A; this is a fake illustration, not real data.)

Figure 3 -- Certificate format, worked example (fake data).

CELLGUARD-10 ENGINEERING PACKET

1. OVERVIEW

A battery pack of two series-connected cells, A and B, is simulated across 320 discrete ticks under a shared charge and thermal controller. Mode, temperature, and fault-latch status are pack-level, shared between both cells. State of charge and terminal voltage are tracked independently per cell -- the two cells have different capacities (manufacturing variance, Figure 1) and can receive different delivered current on any given tick (section 2, cell balancing). Figure 1 lists every fixed model constant. Figure 2 lists the eleven per-tick update rules, in their required order. Initial state, before tick 1: state of charge 0 for both cells, temperature AMBIENT, mode CC, fault not latched, hysteresis counter 0, reference voltage BASE_VOLTAGE for both cells.

2. CHARGE MODE SEQUENCE

The charge mode is CC, then CV, then TAPER, strictly one-way (never reverts), and is shared by both cells. The CC to CV transition is evaluated against the PREVIOUS tick's reported voltage, taken as whichever of the two cells' voltages is higher (the pack regulates to protect against either cell overvoltage) -- never the current tick's own just-computed voltage. On the tick that transition fires, the regulation current is available immediately, that same tick, set to CC_CURRENT. Once in CV mode, a regulation current value is tracked separately from delivered current and decays by CV_DECAY_STEP every tick (floored at zero, never negative) -- this decay is unconditional for every CV-mode tick, not gated by whether a fault is latched, which only zeroes delivered current (section 5), never the regulation current. The CV to TAPER transition check, and that tick's own mode-commanded current, both read the regulation current as it enters the tick, before that same tick's own decay. This applies starting on the very tick the CC-to-CV transition itself fires: mode is already CV by the time that tick reads and decays the regulation current, so there is no grace tick before decay begins -- the first CV tick both uses CC_CURRENT and immediately decays it by CV_DECAY_STEP for the next tick. In TAPER mode, the mode-commanded current (before derating/clamping/balancing) is always the fixed TAPER_CURRENT.

3. THERMAL DERATING AND THE OVERCURRENT CLAMP

If the pack's incoming temperature for a tick is at or above DERATE_TEMP, the mode-commanded current for that tick is multiplied by DERATE_FACTOR -- in every mode, TAPER included -- producing a single, shared BASE current (before either cell's individual balancing adjustment). The clamp step always executes, every tick, AFTER derating, capping the base current at OVERCURRENT_MAX -- but it only changes the value (binds) when the post-derate base current still exceeds OVERCURRENT_MAX. In this model's CC phase, CC_CURRENT (50) exceeds OVERCURRENT_MAX (45), so the clamp binds for as long as derating has not yet engaged; once temperature reaches DERATE_TEMP partway through the CC phase, derating already brings the base current down to 25 before the clamp step runs, so the clamp executes but is a no-op for the remainder of the CC phase. Do not assume the clamp binds uniformly across the whole CC phase -- trace your own engine's incoming-temperature values to find exactly where derating engages.

4. CELL BALANCING

After the clamp (section 3) produces a single shared base current, each cell's OWN delivered current is decided independently. Compute $\text{diff} = \text{soc_a} - \text{soc_b}$ using each cell's state of charge as it entered the tick (before this tick's own update). If the base current is greater than zero and the absolute value of diff is at or above `BALANCE_THRESHOLD`, a passive bleed resistor activates on whichever cell is currently ahead: that cell's delivered current becomes the base current minus `BALANCE_BLEED` (floored at zero); the other, lagging cell receives the full, unmodified base current. If the base current is zero, or the two cells' states of charge are within `BALANCE_THRESHOLD` of each other, both cells receive the same, unmodified base current. This condition is re-evaluated from scratch every single tick -- it is not a one-time decision, and balancing does not stay active for the rest of the run once triggered: it switches on and off repeatedly as the gap crosses the threshold in each direction. Which cell ends up being the one bled is not given to you directly -- it follows from which capacity is smaller (the smaller-capacity cell gains state of charge faster from the same current, so it is the one that pulls ahead and gets bled). Do not assume this without checking the two capacity values yourself, and do not hardcode a cell rather than computing the comparison fresh every tick -- even though, for this packet's fixed capacities, the same cell ends up leading for the whole run.

5. HEAT GENERATION

Heat is a single, shared, pack-level quantity, driven by the shared base current (section 3), not by either cell's individual post-balancing current -- balancing redistributes current between cells, it does not change the total the pack is drawing for thermal purposes. This zero-current rule takes priority over every other heat rule below, including TAPER's: if the base current is zero that tick for ANY reason (fault latched, in any mode including TAPER), heat is zero that tick, full stop -- evaluate this rule first. Only once base current is confirmed nonzero does mode matter: TAPER mode is then an exception to the derating-sensitive heat rule that CC and CV follow -- a nonzero-current TAPER-mode tick generates `HEAT_TAPER` regardless of whether incoming temperature is at or above `DERATE_TEMP` ("always" here means TAPER ignores the temperature check, not that it overrides the zero-current rule above). CC and CV mode ticks with nonzero current follow the general rule: incoming temperature at or above `DERATE_TEMP` generates `HEAT_DERATED`; otherwise `HEAT_FULL`. Temperature is then updated by adding that tick's heat and subtracting `COOL_PER_TICK`.

6. FAULT LATCH

If the pack is not currently fault-latched and the NEW temperature (after this tick's own update) is at or above `FAULT_TEMP_HIGH`, the fault latches immediately and the release-hysteresis counter resets to zero. While latched, both cells deliver zero current every tick (regardless of mode) until released. Release requires the temperature to be at or below `FAULT_TEMP_LOW` for `FAULT_RELEASE_TICKS` CONSECUTIVE ticks -- the counter increments only while temperature stays at or below `FAULT_TEMP_LOW` every tick in a row; a single tick where temperature rises back above `FAULT_TEMP_LOW` resets the counter to zero immediately, even if the pack had been close to releasing. The instant the counter reaches `FAULT_RELEASE_TICKS` and the fault un-latches, the counter itself resets to zero in that same tick.

7. STATE OF CHARGE AND VOLTAGE (PER CELL)

Each cell's state of charge increases each tick by that cell's own delivered current (after balancing, section 4) divided by that cell's OWN capacity (CAPACITY_A or CAPACITY_B -- they differ), times 100, capped at 100. Each cell's terminal voltage is BASE_VOLTAGE plus that cell's own state of charge times VOLT_PER_SOC_PCT, minus that cell's own delivered current times IR_DROP_PER_UNIT -- that cell's own voltage for this tick, which feeds the MAX() in section 2's NEXT-tick mode-transition check. Every state variable (both cells' state of charge, temperature, the CV regulation current, both cells' voltage) carries full, unrounded numeric precision from tick to tick; the certificate's 2-decimal-place rounding (Figure 3) is a display rule applied once at serialization, and never feeds back into the next tick's computation.

8. INDEPENDENT VERIFICATION

Deliver a separately coded verifier that re-implements these eleven rules itself, from scratch, in its own code -- never importing, calling, or reading any state computed by the primary engine. It must check the primary's claimed state at several checkpoint ticks by independently replaying the simulation from tick 1 up to each checkpoint using only its own logic. Run the two required adversarial mutations against it: a claimed fault release after only two consecutive cool ticks instead of four, and a claimed state where both cells report identical delivered current throughout, as if cell balancing were never applied. Confirm the verifier rejects both while still accepting the true trace's own correct checkpoint states.

9. CERTIFICATION

See Figure 3 for the exact, fully mechanical certificate serialization format and a worked example using fake numbers. Deliver six products as files: the engine source; the independent verifier source; the full 320-tick, two-cell state trace; a findings report; certification evidence (both adversarial rejection results plus the SHA-256 trace-integrity hash); and an engineering memo explaining, citing your own executed engine's actual reported values: why TAPER mode never follows the derated-heat rule even while hot; why the fault latch released at the specific tick it did, citing the consecutive-tick hysteresis counter; exactly which CC-phase ticks the overcurrent clamp actually changes the delivered current on, and why it stops changing it partway through; and why cell balancing activates and deactivates repeatedly across the run rather than settling permanently once triggered. Base every reported value on your own executed implementation; never hand-derive or embed a precomputed state, checkpoint value, or hash as a substitute for running the delivered engine.